

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA11

COURSE OUTCOME COVERED

***CO1:** Apply object-oriented concepts and software development life cycle models to design structured and modular software systems. (BL2, BL3)*

Assessment Tool Weightage: 60%

Dr Manimaran

QUESTIONS: 12

Total Marks:60

Annexure A

CONCEPT MAPPING

Code of Conduct:

I Saranraj.G (192511182) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

CONCEPT MAPPING

Question 1. Construct a Concept Map

Given:

Requirement Analysis → _____ → Coding → Testing → _____ → Maintenance

Answer:

Concept Map:

Requirement Analysis → **Design** → Coding → Testing → **Deployment** → Maintenance

The missing concepts are:

- Design
- Deployment

Explanation:

The Software Development Life Cycle (SDLC) is a systematic process used to develop high-quality software. After the Requirement Analysis phase, the **Design** phase creates the system architecture, database design, user interface, and overall software structure based on user requirements. Developers then write the program during the **Coding** phase.

Once coding is completed, the software is verified during the **Testing** phase to identify and correct defects. After successful testing, the software is released to end users through the **Deployment** phase. Finally, the software enters the **Maintenance** phase, where bugs are fixed, updates are made, and new features are added to improve the system.

Analysis:

- Design converts user requirements into a detailed software blueprint.
- Deployment makes the software available for end users.
- Following the SDLC ensures systematic and organized software development.
- Each phase minimizes project risks and improves software quality.
- Proper execution of all phases results in reliable, maintainable, and scalable software.

Question 2. Construct a Concept Map

Given:

Object-Oriented Principles

→ Abstraction

→ _____

→ Inheritance

→ _____

Answer:

Concept Map:

Object-Oriented Principles

- Abstraction
- **Encapsulation**
- Inheritance
- **Polymorphism**

The missing concepts are:

- Encapsulation
- Polymorphism

Explanation:

Object-Oriented Programming (OOP) is based on four core principles that help in designing efficient and reusable software systems.

Abstraction hides unnecessary implementation details while exposing only essential features. **Encapsulation** combines data and methods into a single unit called a class and protects data from unauthorized access using access modifiers. **Inheritance** enables one class to acquire the properties and behaviors of another class, reducing code duplication. **Polymorphism** allows a single interface or method to perform different tasks depending on the object, improving flexibility.

Analysis:

- Encapsulation improves data security and integrity.
- Inheritance promotes code reuse and reduces development effort.

- Polymorphism provides flexibility through dynamic behavior.
- Abstraction simplifies complex systems.
- Together, these principles improve modularity, maintainability, and scalability.

Question 3. Construct a Concept Map

Given:

Class → Blueprint

Object → Instance

Method → _____

Attribute → _____

Answer:

Concept Map:

- Class → Blueprint
- Object → Instance
- Method → **Behavior**
- Attribute → **State**

The missing concepts are:

- Behavior
- State

Explanation:

In Object-Oriented Programming (OOP), a **Class** is a blueprint or template used to create objects. It defines the attributes (data members) and methods (functions) that every object created from it will possess.

An **Object** is an instance of a class that occupies memory and represents a real-world entity.

A **Method** defines the **behavior** or actions that an object can perform. Examples include displaying information, calculating values, or updating records.

An **Attribute** represents the **state** of an object by storing its characteristics or properties, such as name, age, ID, or salary.

Together, classes, objects, methods, and attributes form the foundation of object-oriented system design.

Analysis:

- Classes help create reusable software components.
- Objects represent real-world entities within a program.
- Methods define the functionality of an object.
- Attributes store the object's data and current state.
- These concepts improve modularity, maintainability, and code organization.

Question 4. A Concept Map Shows

Given:

Software Quality

→ Reliability

→ _____

→ Maintainability

Answer:

Concept Map:

Software Quality

- Reliability
- **Usability**
- Maintainability

The missing concept is:

- Usability

Explanation:

Software quality refers to the degree to which a software product meets user expectations and performs efficiently.

Reliability is the ability of software to function correctly without failures for a specified period under given conditions.

Usability measures how easy it is for users to learn, understand, and operate the software. A user-friendly interface improves productivity and reduces user errors.

Maintainability refers to the ease with which software can be modified to fix bugs, improve performance, or add new features after deployment.

These quality attributes work together to ensure the software is dependable, efficient, and easy to maintain throughout its lifecycle.

Analysis:

- Reliability increases user confidence in the software.
- Usability improves customer satisfaction and user experience.
- Maintainability reduces maintenance cost and effort.
- High-quality software enhances business value and customer acceptance.
- These attributes contribute to the long-term success of software systems.

Question 5. UML Concept Map

Given:

UML

→ Structural Diagrams → _____

→ Behavioral Diagrams → Activity Diagram

→ Interaction Diagrams → _____

Answer:

Concept Map:

UML

- Structural Diagrams → **Object Diagram**
- Behavioral Diagrams → Activity Diagram
- Interaction Diagrams → **Communication Diagram**

The missing concepts are:

- Object Diagram

- Communication Diagram

Explanation:

The **Unified Modeling Language (UML)** is a standardized visual language used to model, design, and document software systems. UML diagrams are categorized into structural, behavioral, and interaction diagrams.

An **Object Diagram** is a structural diagram that represents the objects of a system at a specific moment and shows the relationships between those object instances.

An **Activity Diagram** is a behavioral diagram that illustrates the flow of activities, decision points, and business processes within a system.

A **Communication Diagram** is an interaction diagram that focuses on how objects exchange messages with one another to accomplish specific tasks.

Using UML diagrams helps developers understand system architecture, improve communication among stakeholders, and simplify software design.

Question 6. Given**Given:**

Requirements ✓ → Design ✓ → Coding ✗ → Testing ✗ → Deployment ✗

Answer:**Failure Point:****Coding Phase****Explanation:**

The failure occurs during the **Coding** phase because the software design is incorrectly translated into source code. Coding errors may include syntax errors, logical mistakes, incorrect algorithms, improper use of variables, or failure to follow programming standards. Since the program is not implemented correctly, the software cannot proceed successfully to the Testing phase.

As coding is the foundation for testing, any defects introduced during implementation prevent effective testing and delay deployment. Early detection of coding errors through reviews and debugging significantly improves software quality.

Analysis:

- Coding errors lead to incorrect software functionality.

- Defects introduced during coding increase debugging effort.
- Testing cannot be completed successfully until coding issues are resolved.
- Project cost and development time increase due to rework.
- Following coding standards ensures reliable and maintainable software.

Solution:

- Review and debug the source code thoroughly.
- Follow coding standards and best programming practices.
- Perform unit testing after completing each module.
- Conduct peer code reviews to identify errors early.
- Use version control and continuous integration for better quality management.

Question 7. Construct a Concept Map

Given:

Problem Identification → _____ → Design → Implementation → _____

Answer:

Concept Map:

Problem Identification → **Requirement Analysis** → Design → Implementation → **Testing**

The missing concepts are:

- Requirement Analysis
- Testing

Explanation:

The software development process begins with **Problem Identification**, where the problem and project objectives are clearly understood.

The next phase is **Requirement Analysis**, where developers gather user needs, functional requirements, and system constraints. These requirements are transformed into a detailed **Design**, which serves as the blueprint for development.

During **Implementation**, developers write the source code based on the design. Finally, **Testing** verifies that the developed software satisfies user requirements, identifies defects, and ensures the system functions correctly before deployment.

Analysis:

- Requirement Analysis ensures accurate understanding of customer needs.
- Design provides a structured plan for software development.
- Testing verifies software quality and reliability.
- Following these stages reduces development errors and project risks.
- A systematic process improves software quality and customer satisfaction.

Question 8. Construct a Concept Map

Given:

Object-Oriented Features

→ Class

→ _____

→ Message Passing

→ _____

Answer:

Concept Map:

Object-Oriented Features

- Class
- **Object**
- Message Passing
- **Dynamic Binding**

The missing concepts are:

- Object
- Dynamic Binding

Explanation:

Object-Oriented Programming (OOP) consists of several important features that improve software design.

A **Class** is a blueprint that defines the attributes and methods of an object.

An **Object** is an instance of a class that stores data and performs operations defined by the class.

Message Passing allows objects to communicate with one another by invoking methods and exchanging information.

Dynamic Binding is the process of determining which method should be executed at runtime, supporting runtime polymorphism and making programs more flexible.

Analysis:

- Objects represent real-world entities.
- Message passing enables effective communication between objects.
- Dynamic binding supports runtime flexibility.
- These features improve code reusability and maintainability.
- Object-oriented features simplify software development and future enhancements.

Question 9. Construct a Concept Map**Given:**

Software Development Models

→ Waterfall

→ _____

→ Spiral

→ _____

Answer:**Concept Map:**

Software Development Models

- Waterfall
- **Incremental Model**

- Spiral
- **Agile Model**

The missing concepts are:

- Incremental Model
- Agile Model

Explanation:

Software Development Models define the methodology used to develop software projects.

The **Waterfall Model** follows a sequential approach where each phase is completed before moving to the next.

The **Incremental Model** develops software in smaller functional modules, allowing users to receive working software at regular intervals.

The **Spiral Model** combines iterative development with continuous risk analysis, making it suitable for large and complex projects.

The **Agile Model** focuses on iterative development, continuous customer feedback, and rapid delivery of software in short development cycles called sprints.

Analysis:

- Different models are suitable for different project requirements.
- Incremental development reduces project risk.
- Agile improves customer involvement and adaptability.
- Spiral effectively manages high-risk projects.
- Selecting an appropriate model improves software quality and project success.

Question 10. Construct a Concept Map

Given:

Class Diagram

→ Class

→ _____

→ Association

→ _____

Answer:

Concept Map:

Class Diagram

- Class
- **Attribute**
- Association
- **Method**

The missing concepts are:

- Attribute
- Method

Explanation:

A **Class Diagram** is one of the most important UML structural diagrams used to represent the static structure of a software system.

A **Class** represents a blueprint that defines the characteristics and behavior of objects.

An **Attribute** defines the properties or data members of a class, such as name, ID, or age.

An **Association** represents the relationship between two or more classes, showing how they interact with each other.

A **Method** defines the operations or functions performed by the objects of a class.

Class diagrams help developers understand system structure and relationships before implementation.

Analysis:

- Class diagrams provide a clear representation of system structure.
- Attributes define the data stored within a class.
- Methods represent the behavior of objects.
- Associations illustrate relationships among classes.
- Class diagrams improve software documentation, design, and maintainability.

Question 11. Construct a Concept Map

Given:

Software Engineering Principles

→ Modularity

→ _____

→ Reusability

→ _____

Answer:

Concept Map:

Software Engineering Principles

- Modularity
- **Maintainability**
- Reusability
- **Scalability**

The missing concepts are:

- Maintainability
- Scalability

Explanation:

Software Engineering Principles are guidelines that help developers design high-quality, reliable, and efficient software systems.

Modularity divides a software system into smaller, independent modules that can be developed, tested, and maintained separately.

Maintainability refers to the ease with which software can be modified to fix defects, improve performance, or add new features after deployment.

Reusability allows existing software components or modules to be used in multiple applications, reducing development time and cost.

Scalability is the ability of a software system to handle increasing workloads, users, or data without significant performance degradation.

Applying these principles results in software that is flexible, reliable, and easy to manage throughout its lifecycle.

Analysis:

- Modularity simplifies software development and testing.
- Maintainability reduces the cost and effort required for software updates.
- Reusability minimizes code duplication and accelerates development.
- Scalability enables software to support future growth and increased demand.
- These principles improve software quality, performance, and long-term sustainability.

Question 12. Given

Given:

Requirement Analysis ✓ → System Design ✓ → Coding ✓ → Integration ✗ → Deployment ✗

Answer:

Failure Point:

Integration Phase

Explanation:

The failure occurs during the **Integration** phase, where individually developed modules are combined into a single software system. Although each module may function correctly on its own, problems can arise when they interact with one another.

Common integration issues include incompatible module interfaces, incorrect data exchange, communication failures, API mismatches, dependency conflicts, and configuration errors. These issues prevent the complete system from functioning correctly, making deployment impossible until the problems are resolved.

Proper integration testing ensures that all modules communicate effectively and work together as expected.

Analysis:

- Integration failures cause functional inconsistencies between software modules.
- Communication and interface mismatches increase debugging effort.
- Deployment is delayed until integration issues are resolved.
- Early integration testing reduces project risk and improves software quality.
- Effective integration ensures a stable, reliable, and fully functional software system.

Solution:

- Perform comprehensive integration testing after combining modules.
- Verify module interfaces and communication protocols.
- Resolve compatibility and dependency issues before deployment.
- Use continuous integration tools to detect integration problems early.
- Maintain proper documentation and standardized interfaces to ensure smooth module integration.

CONCEPT MAPPING

RUBRICS FOR CALCULATION

CONCEPT MAPPING

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

